

Activation Planner

Reference document for a new, standalone project discussed alongside IcomRigControl. Compiled to keep ideas, decisions, and open items straight across sessions.

1. Origin, Purpose, and Naming

This project grew out of a broader brainstorm of ham radio coding project ideas. Jim initially asked to expand specifically on the idea of a Field Day / POTA activation planning tool. The core goal: a tool that helps plan an operating session ahead of time — which bands are likely to work, which antenna to bring, and what gear to pack — grounded in real propagation science rather than guesswork.

NAMING DECIDED

Jim pointed out that everything being designed — propagation prediction, antenna selection, gear checklists, spotting — is genuinely useful for any ham radio operating session, not just POTA specifically: Field Day, SOTA, a casual rag chew, or any portable or home-station operation could all benefit equally. The project was renamed from 'POTA Activation Planner' to simply **Activation Planner** to reflect this. POTA-specific features (like posting spots to the POTA website, Section 6) remain part of the plan as one supported use case among several, not the sole purpose of the tool.

2. Standalone vs. Integrated Decision

DECIDED

Claude laid out a pros/cons analysis of building this as a new panel inside IcomRigControl versus a standalone application. Jim chose **standalone**.

Reasoning for standalone (from the analysis):

- Clean separation: planning is pre-trip, IcomRigControl is in-the-moment operation.
- Freedom to design its own UI/UX (maps, checklists, trip-prep flows) without being constrained by IcomRigControl's existing layout or its pending redesign question.
- Lower risk to the stable, working IcomRigControl codebase.
- More natural to eventually share or open-source than a slice of a personal rig-control app.

Suggested design boundary (Claude's recommendation, not yet explicitly confirmed by Jim): the planner does **only** planning — location input to band/antenna/checklist output — and does NOT duplicate QSO logging. Actual activation logging stays in IcomRigControl, which already does it well (ADIF export, contest mode, LoTW, etc.).

3. Technology Stack

DECIDED

Jim confirmed: build using **C# and .NET 10**, matching IcomRigControl's stack. Cross-platform support (Windows, macOS, Linux) was explicitly requested. UI framework (Avalonia, matching IcomRigControl, or something else) has not yet been explicitly decided.

4. Propagation Prediction (VOACAP)

YOUR IDEA + RESEARCHED

Jim's original request: band suggestions should be based on real scientific propagation data, not just rules of thumb, and antenna recommendations should be based on predicted propagation matched against a user-entered antenna inventory.

4.1 Important finding: VOACAP Online prohibits automation

The official VOACAP Online website explicitly states that all automated access via scripts or programs is strictly prohibited unless pre-arranged with the site owner, with an immediate and permanent ban for violations. This rules out simply scripting calls to voacap.com.

4.2 The legitimate path: standalone VOACAP, run locally

VOACAP itself is free, open software with real standalone versions for all three target platforms:

VOACAPW.EXE (Windows), **voacapl** (Linux, community port by James Watson), and an OSX build.

Running a program the user has installed locally, and shelling out to it via `Process.Start`, is legitimate — the same pattern already proven in `IcomRigControl` for TQSL (LoTW signing), `afplay` (macOS audio playback), and `arecord` (Linux audio, discussed for a related feature). The user would install VOACAP once, similar to the existing TQSL requirement for LoTW.

4.3 Licensing: does using VOACAP affect this project's own license?

Jim asked: if the planner is licensed under AGPLv3 or GPLv3, and VOACAP is open source, can VOACAP be incorporated into the program? Researched directly.

VOACAP is written in **Fortran** (the original NTIA/Institute for Telecommunication Sciences source; ported to Linux via GFortran as `voacapl`). Every source describes the Fortran source code as 'freely available for downloading' from NTIA/ITS, a U.S. government agency — but this has not been confirmed to carry a formal open-source license (MIT/GPL/AGPL/etc.) with an explicit LICENSE file or declaration. This should be verified directly against the actual downloaded source archive, or with NTIA/ITS, before assuming any specific compatibility.

This uncertainty turns out not to matter for the planner, because the plan was never to merge or link VOACAP's source code into the C# project at all. The design already settled on in Section 4.2 — writing an input text file, launching the separately-installed VOACAP as an external process via `Process.Start`, and reading its text output — means the planner never distributes, links, or embeds any of VOACAP's code. The end user installs real VOACAP themselves, directly from NTIA/ITS, under whatever terms NTIA/ITS provides. This is 'mere aggregation' / independent-process interoperation, the same relationship the planner has with any other external command-line tool it shells out to (`afplay`, `arecord`, TQSL) — not linking or embedding, which is treated very differently, and much more simply, under essentially every open-source license including AGPLv3 and GPLv3. This is also exactly how existing tools like `fldigi` and `HamCAP` already relate to VOACAP.

CONCLUSION: Jim's choice of AGPLv3 or GPLv3 for his own planner code is unaffected by VOACAP's licensing, because the two programs never share or link code — the planner calls VOACAP the same way it would call any separate command-line tool the user has installed. This should still be sanity-checked against VOACAP's actual license terms once located, but the shell-out architecture already chosen is the same one that would be safe under nearly any license VOACAP might carry.

4.4 Input file format ('cards' / 'the deck')

VOACAP's input and output are plain text files, historically called 'cards,' collectively 'the deck.'

Command-line invocation is simple: `voacapl ~/itshfbc infile.dat outfile.out` (Linux/macOS) or `VOACAPW.EXE directory inputfile outputfile` (Windows). A real example deck:

```
COMMENT ...
LINEMAX 55
COEFFS CCIR
TIME 1 24 1 1
MONTH 2003 7.00
SUNSPOT 63.
LABEL VAASA NEW YORK
CIRCUIT 63.10N 21.60E 40.72N 74.00W S 0
SYSTEM 1. 155. 3.00 90. 24.0 3.00 0.10
FPROB 1.00 1.00 1.00 0.00
ANTENNA 1 1 2 30 0.000[default\CCIR.000 ] 0.0 0.1000
ANTENNA 2 2 2 30 0.000[default\SWWHIP.VOA ] 0.0 0.0000
FREQUENCY 7.0510.1214.05 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00
METHOD 30 0
EXECUTE
QUIT
```

Key cards relevant to this project: TIME (hour range for the activation window), MONTH + SUNSPOT (date and real solar activity number, to be pulled from NOAA SWPC rather than guessed), CIRCUIT (transmit and receive lat/lon — the park as transmit side, representative listening locations as receive side), ANTENNA (references antenna model files — see 4.5 below), and FREQUENCY (up to 11 frequencies per run, with column-exact fixed formatting).

Important warning found directly in VOACAP's own documentation: 'The formatting of the input deck is critical to the operation of voacap and many bugs can be traced to a malformed deck.' The C# writer for this format needs careful, tested, fixed-column formatting — the same discipline already used successfully in IcomRigControl for AX.25 bit-level encoding and APRS's exact-width position strings.

4.5 Antenna modeling

Custom antenna files ('type-13' format) are genuinely complex: a full 360-degree, 1-degree-increment, all-elevation-angle radiation pattern, roughly 260KB of data per antenna per frequency, normally generated by dedicated antenna-modeling software (NEC/MININEC/EZNEC/HFANT) — not something to hand-write from a simple text description of an antenna.

Jim asked directly whether antenna modeling could genuinely be incorporated into the program, since it affects both propagation prediction and antenna selection. Researched further and found a real, buildable path — not just a fallback.

Option A (fallback, always available): community antenna-category library

VOACAP ships with pre-built standard antenna models, and there is a real community-curated collection specifically for amateur band antennas — covering dipoles/doublets at various heights (35', 55', 75', 95'), end-fed wires, verticals, square loops, and phased arrays, evaluated across the actual amateur bands. These are realistic matches for common POTA antennas (EFHW, whip/vertical, wire dipole). Approach: maintain a small internal library mapping common antenna *categories* (vertical/whip, EFHW, dipole-at-height-X, magnetic loop) to the closest matching pre-built type-13 file, and tag each gear inventory entry with its closest category. Simple, no new dependency, but only an approximate match to Jim's actual

antennas.

Option B (real incorporation): NEC2++ for true custom modeling

NEC2++ is a real, actively maintained, **GPL v2 licensed** open-source electromagnetic simulation engine, compatible with the standard NEC-2 antenna description format. Unlike VOACAP's Fortran origins, NEC2++ is a proper C++ rewrite, explicitly built to be embedded in other tools — it ships both as a standalone command-line executable AND as a callable C/C++/Python/Ruby library, specifically described by its own documentation as suited for 'GUI tools and automatic antenna optimization systems.' It is genuinely cross-platform.

Two real levels of integration are possible:

- **Shell-out (consistent with everything else in this plan):** write a NEC-2 geometry file describing an antenna Jim actually owns (wire length, height above ground, ground quality), run the necpp executable as an external process (same Process.Start pattern as VOACAP/TQSL/afplay/arecord), and parse its gain/pattern output into a real, custom VOACAP type-13 antenna file — genuinely matching VOACAP's predictions to Jim's specific gear, rather than an approximate category.
- **True in-process integration (more ambitious):** P/Invoke NEC2++'s C library directly from C#, avoiding external-process overhead and enabling real-time, interactive modeling — e.g. a live-updating radiation pattern as antenna height is adjusted via a UI slider. Real native-interop work, but NEC2++'s C interface is far more standard and approachable than the raw AVFoundation work considered (and deferred) for macOS audio recording.

RECOMMENDATION (Claude's, not yet confirmed by Jim): start with Option A (community library) to get the planner working end-to-end quickly, then add Option B (NEC2++ shell-out) as a genuine enhancement — when a user's antenna doesn't match a standard category well, or for precise modeling of a specific known setup (e.g. a particular wire length and height combination), generate a true custom model instead of approximating. This keeps the MVP simple while leaving a real, scientifically grounded path to the deeper feature Jim actually asked for.

5. GPS Input

YOUR IDEA + RESEARCHED

Jim asked whether GPS input would help. Confirmed: yes, in two concrete ways.

- Auto-fill the CIRCUIT card's transmit coordinates directly from a connected GPS (or phone location, or the IC-705's built-in GPS, researched separately for IcomRigControl) instead of manual entry.
- Auto-detect which POTA park the user is actually at, by cross-referencing GPS coordinates against POTA's park boundary database — removing manual park-reference entry error.

.NET has solid cross-platform geolocation APIs; this is considered low-effort to add and worth building in from the start rather than bolting on later.

6. POTA Website Integration

YOUR IDEA

Jim requested the ability to report to the POTA website (spotting info and other data POTA collects), noting several existing web-based POTA programs already do this. Identified real, concrete pieces:

- Self-spotting: POTA has a documented API for posting the operator's own spot (frequency, mode, callsign) so hunters can find them.
- Pulling the live spot feed: useful context even at planning time (seeing what other activators/hunters are doing nearby).
- Reading back park info: POTA's API exposes park metadata (boundaries, activation history, prior activator counts), which could feed into planning — e.g. flagging a park that's never been activated on a particular band.

NOT yet researched in detail: the exact POTA API endpoints, authentication requirements, and rate limits. This should be looked into properly before building, the same way VOACAP's automation policy was checked before assuming it was fine to script.

7. Gear Inventory and Checklist

YOUR IDEA

Jim's request: let the user enter their full ham gear inventory, have the program compile a checklist of items to bring with checkboxes, and allow the list to be freely edited (add/remove items).

7.1 Data model sketch (Claude's suggestion)

Gear Item (reusable inventory entry): name, category (Antenna, Radio, Power, Logging, Comfort/Safety, Cables/Adapters, Other), optional notes, and a flag for whether it's a default item (auto-included every time) versus situational.

Checklist Template vs. Checklist Instance: a Template is a standing, reusable loadout (e.g. 'Full Field Day kit,' 'Minimalist POTA,' 'EMCOMM go-bag'). An Instance is a specific activation's actual checklist, started from a template, then freely edited for that trip, tracking its own check-off state independently without altering the template. This mirrors the ContestDefinition-vs-session-state pattern already used successfully in IcomRigControl's contest mode.

7.2 Suggested features (Claude's suggestions, not yet confirmed)

- Multiple templates for different activity types.
- Check-off state persists if packing is interrupted and resumed later.
- A warning if an item marked essential isn't checked before finalizing a trip.
- A quick 'reset for next time' to uncheck everything in an instance without deleting it.
- Free-form add-on-the-fly during packing, not limited to template items.

8. Digital Interface Hardware Jim Owns

OPEN QUESTION

Jim owns three pieces of hardware relevant to digital modes and packet/APRS operation, raised late in the conversation and not yet placed into the plan for either IcomRigControl or the Activation Planner.

8.1 Mobilinkd TNC4

A Bluetooth/USB hardware AX.25/APRS TNC (terminal node controller) — performs AX.25/APRS modulation in dedicated hardware rather than software.

- Whether/how it relates to IcomRigControl's existing software AFSK modem (Phase 10) — as an alternative hardware modulation path, or a complementary option.
- Whether it belongs in the Activation Planner's gear checklist as an inventory item, and/or as a supported device for spotting/logging while activating.

8.2 Digirig Mobile

A small USB soundcard + CAT/PTT interface, popular for digital modes and portable/POTA operation. Bridges a radio's audio and PTT/CAT control to a computer over USB.

8.3 Signalink USB

A USB soundcard interface serving a similar core purpose to the Digirig — connecting a radio's audio (and typically PTT) to a computer for digital modes.

8.4 Open questions common to all three

- Jim owns both a Digirig Mobile and a Signalink USB, which serve an overlapping purpose (radio audio/PTT interface for digital modes). Worth clarifying whether each is dedicated to a different station/setup (e.g. one for a base station, one for a portable/POTA kit), or whether one is a spare/backup for the other.
- None of the three devices have yet been connected to IcomRigControl's Phase 10 APRS/audio pipeline (NAudioPlayer/MacAudioPlayer) or to the planned Activation Planner's gear checklist — this needs a deliberate design pass, not a reactive addition.
- Whether IcomRigControl's Settings audio-device dropdown (built in Phase 10) would already show the Digirig/Signalink as selectable output devices on Windows, since they present as standard USB audio devices to the OS — likely yes, but not yet confirmed against real hardware.

ACTION ITEM: revisit this specifically in a future session once the core planner architecture is settled, and decide deliberately rather than bolting it on reactively. Likely worth testing whether IcomRigControl's existing audio device selection already recognizes the Digirig/Signalink as a usable output device before assuming new integration work is needed.

9. EMCOMM Use Case

YOUR IDEA

Jim flagged that EMCOMM (emergency communications) should not be forgotten as a use case for the Activation Planner. This is a recurring, genuine thread across Jim's work — his StarCom net involvement, and the EMMCOM Field Comms Server integration already built into IcomRigControl (Phase 6, complete) — and the tool's rename away from a POTA-specific name makes EMCOMM an explicit, legitimate use case rather than an afterthought.

9.1 What EMCOMM genuinely needs beyond the base design

- **Go-kit checklists with EMCOMM-specific items:** a natural extension of the gear checklist template system already designed (Section 7) — an 'EMCOMM go-bag' template with items like first aid supplies,

extra/redundant power, ICS forms, and priority traffic supplies, alongside the radio gear templates already planned.

- **Propagation planning for NVIS/regional nets rather than DX:** VOACAP genuinely supports this — it is not solely a DX tool — but the 'receive location' side of a prediction would need to represent a served agency or net control station's location rather than distant hunters/DX targets, which is a real, different framing of the same underlying CIRCUIT card mechanism already researched (Section 4.4).
- **Multiple operating positions / team coordination:** EMCOMM activations are often not solo the way a typical POTA activation is. This may require the planner to support a notion of a team roster or assigned operating positions — genuinely new territory, not covered by anything designed so far for the single-operator case.

9.2 Open question: relationship to existing EMMCOM/IcomRigControl work

Not yet decided: should the Activation Planner talk to or complement Jim's existing EMMCOM Field Comms Server (already built into IcomRigControl, Phase 6), or stay deliberately separate — the same way QSO logging was deliberately kept in IcomRigControl rather than duplicated into this new tool (Section 2)? This is a genuine design decision, not yet made, and should be considered deliberately once the core Activation Planner architecture exists, rather than assumed either way.

10. Context: Where This Came From

This planner was one idea among a larger brainstormed list of ham radio coding projects (from an earlier session, 'Ham radio project ideas,' July 10). Other ideas from that list and later brainstorming, for reference and NOT currently in progress: QRP kit building, antenna projects, SDR/panadapter projects, WSPR, JS8Call, Pi-based APRS igate/digipeater, satellite work, NOAA weather satellite reception, a propagation-aware operating assistant using Jim's own logged QSO data, a CW decoder/trainer, a grid-square/DXCC worked/needed tracker with a map, a Morse practice oscillator (could reuse IcomRigControl's AfskModulator tone-generation code), an SWR sweep logger, a 44Net mesh status dashboard, and a voice-to-text QSO logger.

11. Summary: Open Items and Next Steps

OPEN QUESTIONS

- Decide the relationship between the Activation Planner and Jim's existing EMMCOM Field Comms Server / IcomRigControl work (Section 9.2) — integrate, complement, or stay deliberately separate.
- Design the EMCOMM-specific go-kit checklist template and NVIS/regional propagation framing (Section 9.1) once the base checklist and propagation features exist.
- Locate and read VOACAP's actual license terms directly (from the downloaded source archive or NTIA/ITS) to confirm the licensing analysis in Section 4.3 — the shell-out architecture should be safe regardless, but worth confirming.
- Confirm the 'planning-only, no logging duplication' design boundary for the standalone app.
- Decide on a UI framework for the new standalone app (Avalonia again, or something else).
- Research the actual POTA API (endpoints, auth, rate limits) before building spotting/park-lookup features.

- Decide how to place the Mobilinkd TNC4, Digirig Mobile, and Signalink USB in the overall plan (Section 8) — including clarifying their respective roles, since the Digirig and Signalink overlap in purpose.
- Decide whether to build antenna modeling in two stages as recommended (Section 4.5): community category library first (Option A), then NEC2++ shell-out for true custom modeling (Option B) as an enhancement.
- Confirm the antenna-category-mapping approach for VOACAP antenna modeling (Section 4.5) before building Option A.
- Once these are settled: begin architecting the whole app (solution structure, core class library, UI layer), following the same TDD/layered discipline used throughout IcomRigControl.

Legend: **Green** = Jim's idea **Blue** = Claude's suggestion/research **Purple** = Decided together **Orange** = Still open, not yet decided